woven BY TOYOTA

Popping the Hood on Rust

in the 2026 Toyota RAV4

May 19, 2026

Pete LeVasseur

Staff Software Engineer

Technologies & Standards

pete.levasseur@woven.toyota

Lead @ Safety-Critical Rust Consortium



Toyota Shipped Rust in the 2026 Toyota RAV4!



✅ Great! Rust is shipped!

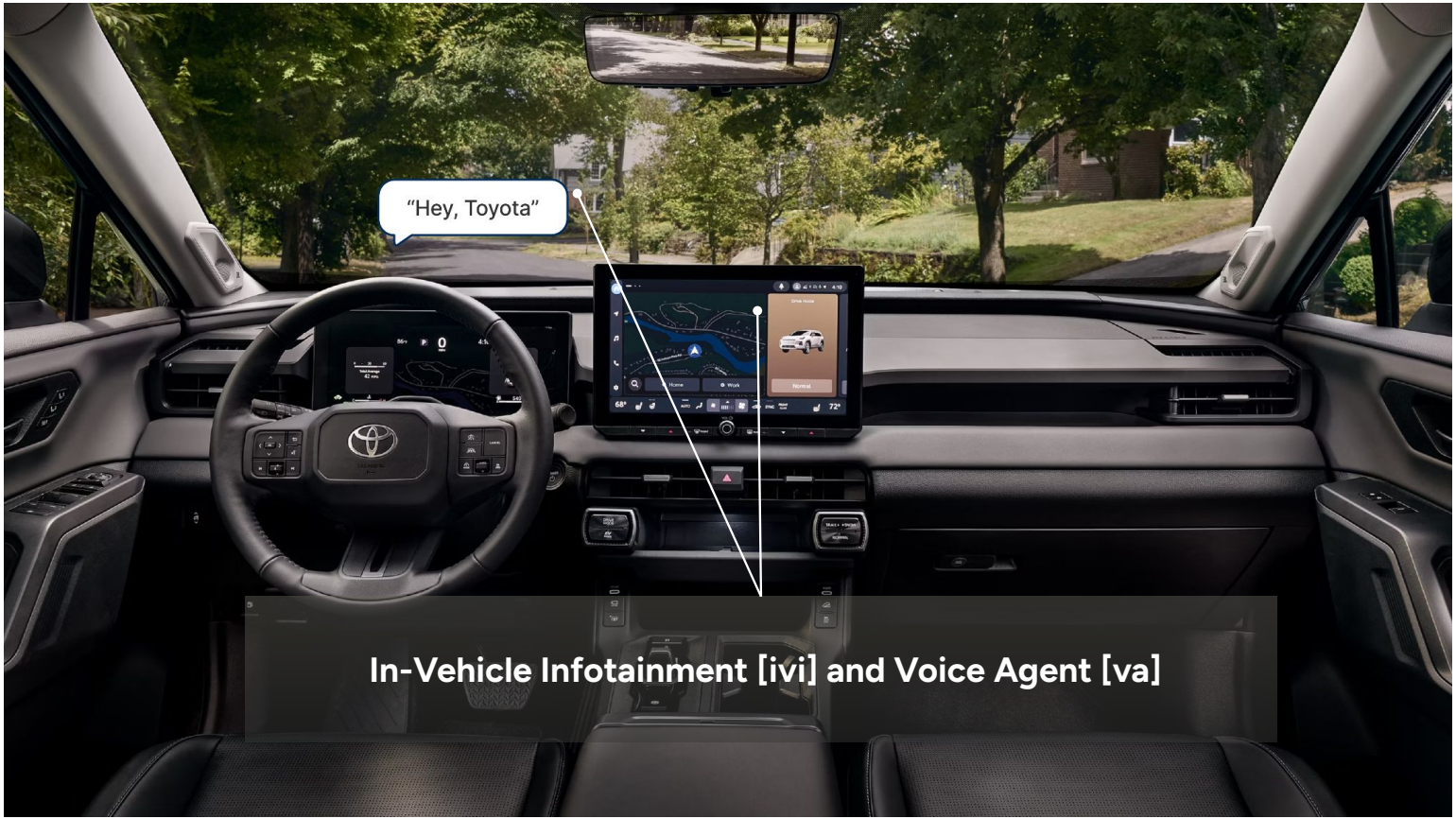
...we must be living in a Rusty utopia?

Choosing Where to Start Matters...

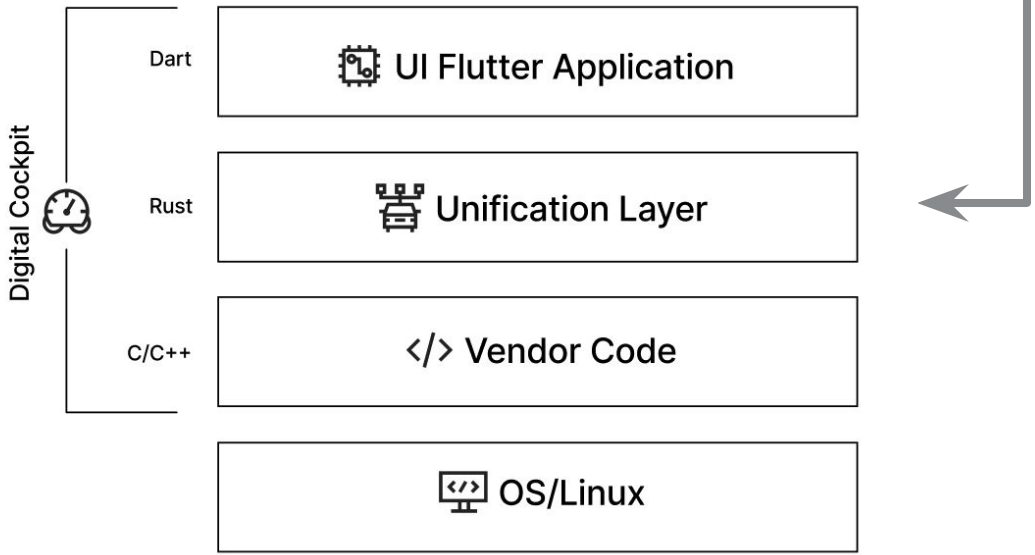
Safety Critical || Digital Cockpit






“Hey, Toyota”
In-Vehicle Infotainment [ivi] and Voice Agent [va]

The Rust Layer: `vehicle_interface`



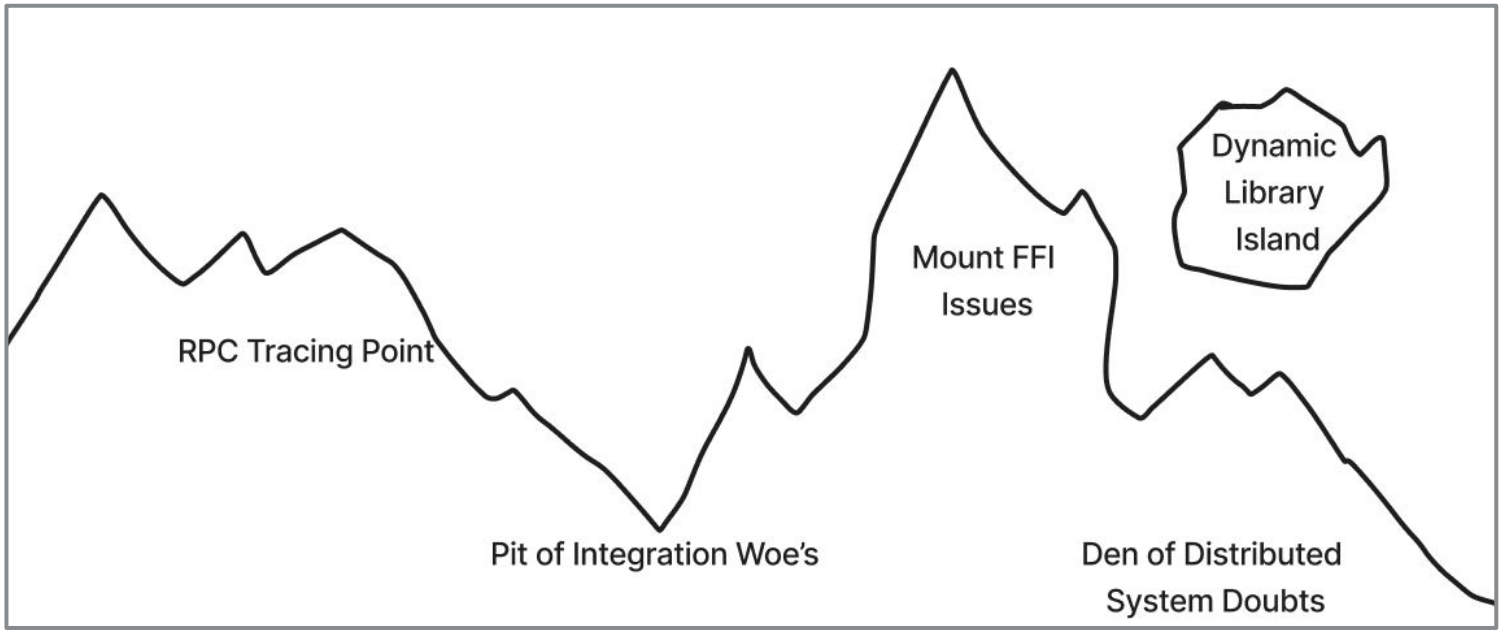
When was the last time we had a panic in our Rust layer?

[could not recall off the top of mind] 🤔

Aik Kalantarian, Software Engineer

Rust as (Safe) Hot Glue

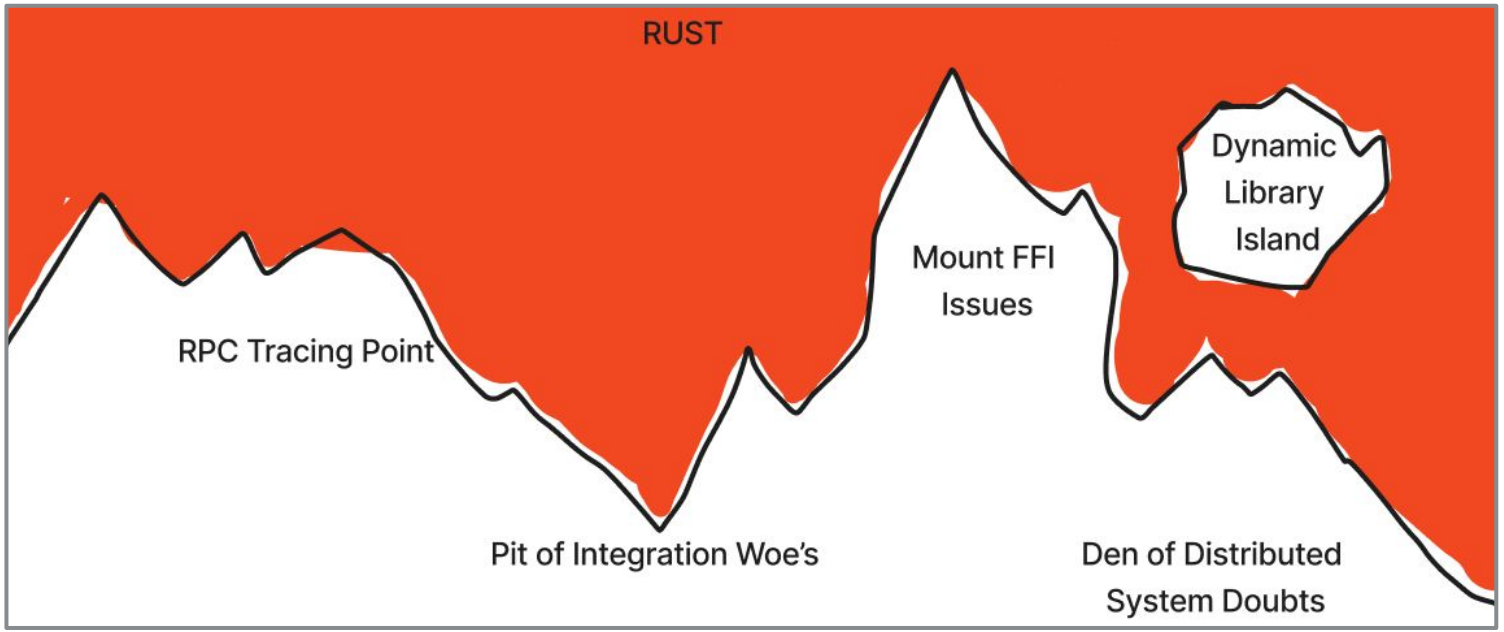
"There is a lot to navigate here"



*API surface from client perspective

Rust as (Safe) Hot Glue

"Look at that consistent API~" 😉

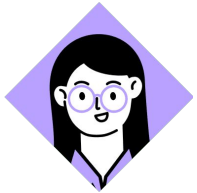


*API surface from client perspective

Rust gave us the **confidence** to build a stable unified API layer **in an evolving software environment** [...] which is not a trivial task.

Jerel Unruh, Principal Engineer

Rust Meets Engineers Where They Are



"Bit twiddling with confidence"

Low Level Systems Engineer



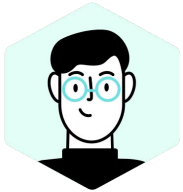
"Copy & paste then learn from the compiler errors"

Rust newbie



"Mux & Proxy async streams with these helpful primitives"

Middleware expert



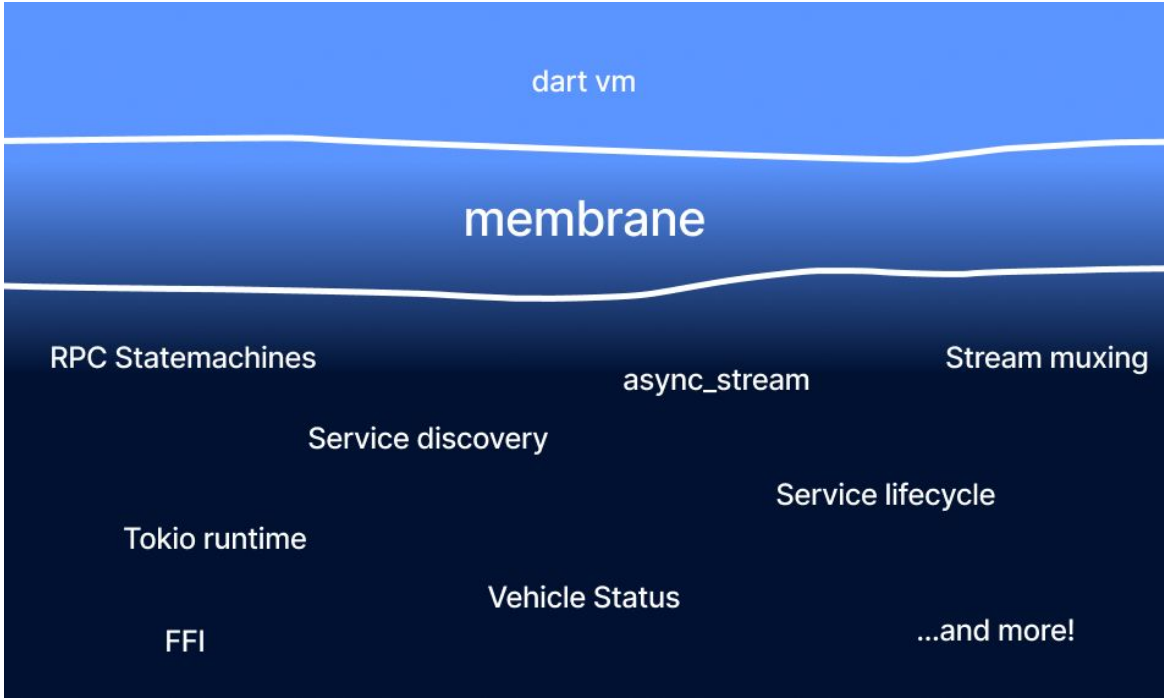
"What's Rust?"

Flutter UI dev

Rust Benefits Escape to Non-Rust Systems!

Rust <=> Dart via **membrane**

Goal: Hide FFI complexity from UI developers using a near-zero copy data ingestion into/out of Dart Virtual Machine



It's opensource! github.com/jerel/membrane

Proc Macros Can Help Non-Rust Developers!

```rust
#[async_dart(namespace = "accounts")]
pub fn contacts() -> impl Stream<Item = Result<data::Contact, data::Error>> {
  futures::stream::iter(vec![Ok(Default::default())])
}

#[async_dart(namespace = "accounts")]
pub async fn contact(id: String) -> Result<data::Contact, data::Error> {
  Ok(data::Contact {
    id: id.parse().unwrap(),
    ..Default::default()
  })
}
```

...now use it from Dart!



```dart
import 'package:dart_example/accounts.dart';

void main(List<String> arguments) async {
  var accounts = AccountsApi();
  print(await accounts.contact(id: "1"));
}
```

Flutter UI devs can grab an API and don’t need to know what the runtime is doing under the hood. This **frees up devs to focus on customer facing features**, instead of chasing down memory leaks and thread locks.

Pete Journell, Principal Engineer

Growing (Excitement/Pains) with Rust

Easy[ier] Language Onboarding

2 Teams Shipping with Rust!

Confidence Inspiring

2 Teams == 2 Rust Versions

600+ Rust Developers

2 Teams == 2 Build Systems

Moving from Cloud to On Device, **we had to mitigate risks**. [...] We needed a modern language with a modern package manager that allowed us to interact with C / C++ Code while mitigating any Undefined Behavior **without raising the bar for hiring.** The answer was clear, it was Rust.

Eric McBride, Principal Engineer

“**The proof is in the progress.** After demonstrating that Rust is an effective tool for creating safe, performant, and reliable in-vehicle software,

Rust is expanding within Toyota from just the frontend integrations to more parts of our stack.”

Kyle Turney, Principal Engineer

Gaining Momentum



Rust is boring in the best way

Memory leaks and thread contention are too exciting, **banish those failure modes into the abyss**, so your teams can focus on the business logic & performance bugs that directly impact customer value



Wider adoption of Rust crates

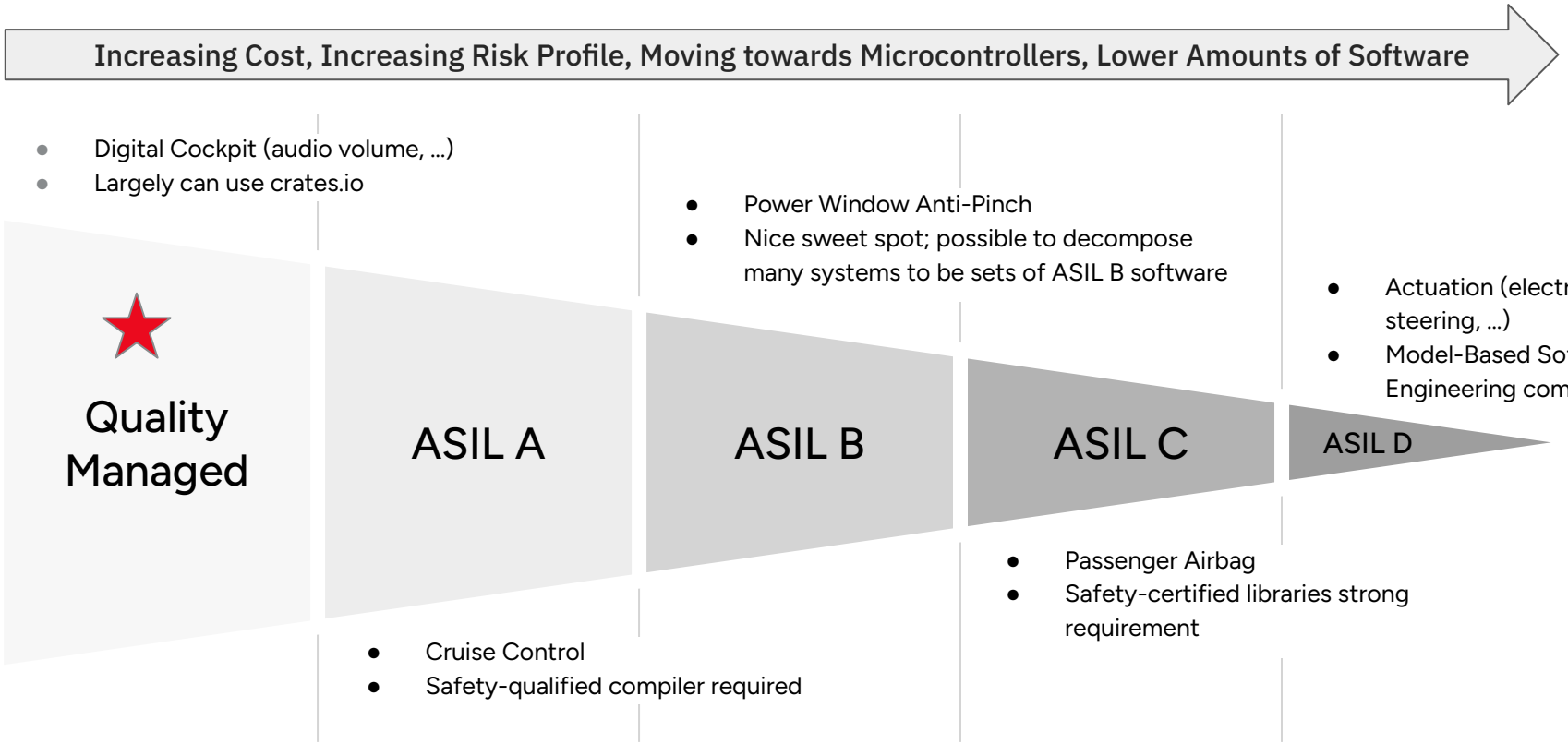
Reduce, reuse, recycle… code. Over and over and over. Teams across Toyota are **sharing more code than ever**. thanks to the crate mechanisms in Rust.



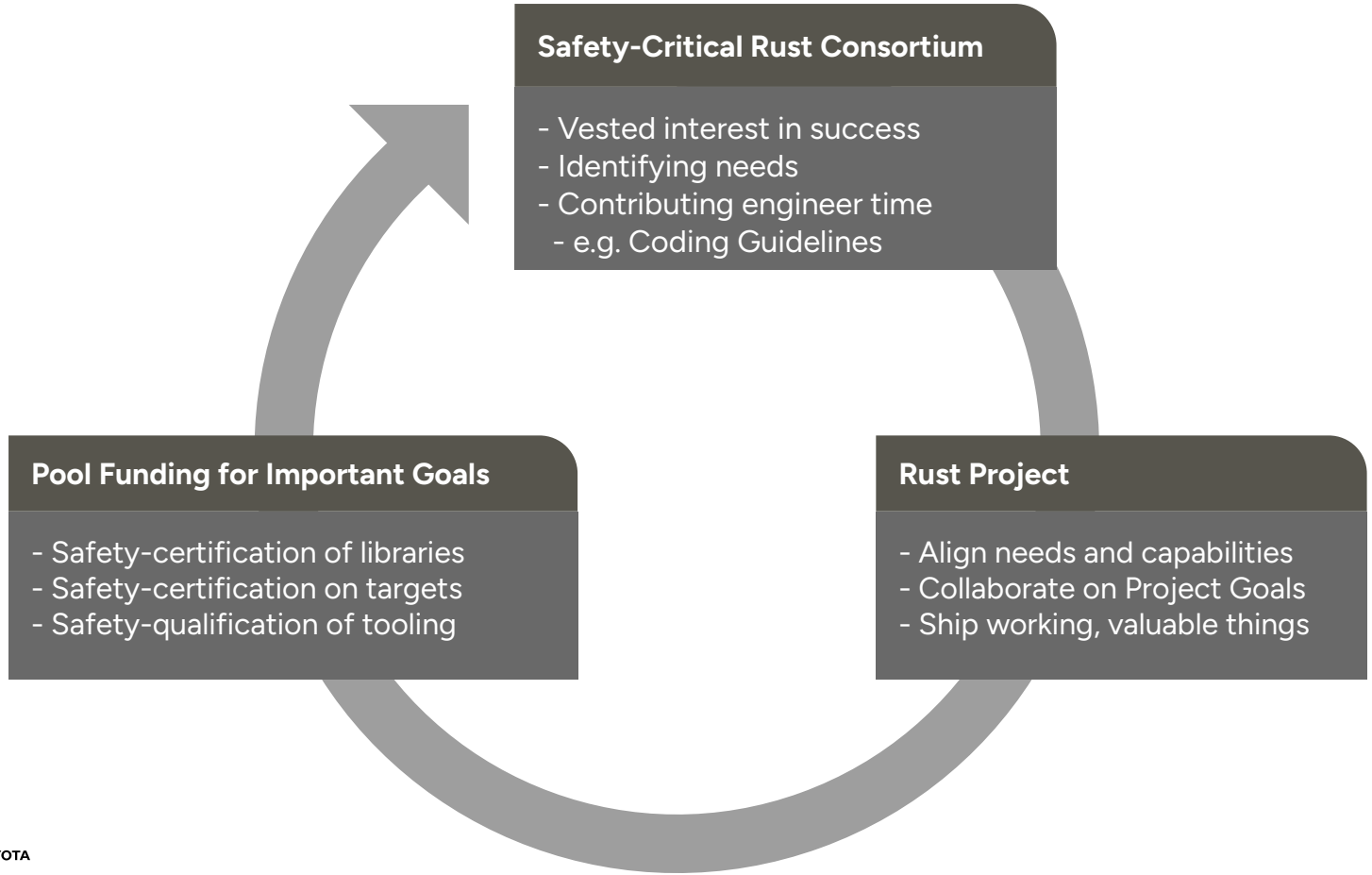
Increasing company confidence

Rust benefits have had measurable impact on real production timelines and can be used in production vehicles. **What comes next?**

Parallel Adoption Paths



Getting Involved to Make Real Change



Growing the Pie







AdaCore

Rust Project Goal in 2026 for adding Modified Condition / Decision Coverage (MC/DC).

Toolchains available; looking for partner to work with on safety-qualification.

Ferrous Systems

Safety-qualified toolchains for ISO 26262, IEC 61508, IEC 62304 are open source.

Safety-certified partial subsets of Rust's core library are open source.

HighTec

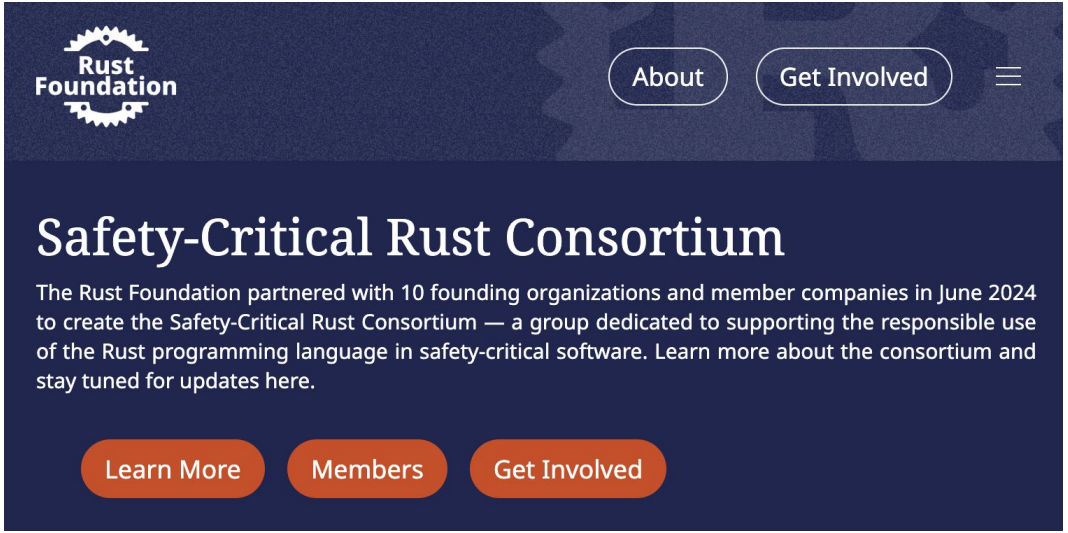
Qualified for ISO 26262

Targets Infineon AURIX MCUs popular in the Automotive industry.

So, are we there yet?

Nope.

But we're good ecosystem partners and walking the path toward the ability to use Rust everywhere on-vehicle.



DOVER, DELAWARE, USA, June 12, 2024 – The Rust Foundation, AdaCore, Arm, Ferrous Systems, HighTec EDV-Systeme GmbH, Lynx Software Technologies, OxidOS, TECHFUND, TrustInSoft, Veecle, and Woven by Toyota are thrilled to jointly announce the Safety-Critical Rust Consortium. The primary objective of this group will be to support the responsible use of the Rust programming language in safety-critical software — systems whose failure can impact human life or cause severe environmental or property harm.

Thank You!

Pete LeVasseur
Staff Software Engineer
E: pete.levasseur@woven.toyota



Safety-Critical Rust Consortium
https://arewesafetycriticalyet.org/

Safety-Critical Rust Consortium

Eclipse SDV WG
https://eclipsesdv.org/

SDV
Eclipse Software Defined Vehicle

Rust Project: FLS
https://rust-lang.github.io/fls/



https://github.com/rustfoundation/safety-critical-rust-consortium
https://github.com/rustfoundation/safety-critical-rust-coding-guidelines